```
import pygame
import os
import math
from config import TILE_SIZE, WIDTH, HEIGHT
from state import STATE

TILE_TYPES = [[illegible]]
TILE_COLORS = {
    '[illegible]': ([illegible]),
    '[illegible]': ([illegible]),
    '[illegible]': ([illegible]),
    '[illegible]': ([illegible]),
    '[illegible]': ([illegible]),
    '[illegible]': ([illegible]),
    '[illegible]': ([illegible]),
    '[illegible]': ([illegible]),
    '[illegible]': ([illegible]),
    '[illegible]': ([illegible]),
    '[illegible]': ([illegible]),
}
ENTITY_TYPES = [[illegible]]

class LevelEditor:
    def __init__(self, map_data=None, map_name="untitled"):
        self.active = False
        self.map_data = map_data
        self.map_name = map_name
        self.current_tile_index = 0
        self.current_entity = "[illegible]"
        self.hover_col = 0
        self.hover_row = 0
        self.font = pygame.font.SysFont("Consolas", [illegible])
        self._built_once = False

    def set_map(self, map_data, map_name="untitled"):
        self.map_data = map_data
        self.map_name = map_name
        self._built_once = False

    def toggle(self):
        if self.map_data is None:
            return
        self.active = not self.active
        if self.active and not self._built_once:
            self.rebuild_world()
            self._built_once = True
        pygame.mouse.set_visible(self.active)

    def handle_event(self, event):
        if not self.active or self.map_data is None:
            return False
        mx, my = pygame.mouse.get_pos()
        cam_x = getattr(STATE, "camera_x", 0)
        cam_y = getattr(STATE, "camera_y", 0)
        world_x = mx + cam_x
        world_y = my + cam_y
        col = int(world_x // TILE_SIZE)
        row = int(world_y // TILE_SIZE)
        if 0 <= row < len(self.map_data) and 0 <= col < len(self.map_data[0]):
            self.hover_col, self.hover_row = col, row
            if event.type == pygame.MOUSEBUTTONDOWN:
                if event.button == 1:
                    self._paint_tile(col, row, rebuild=True)
                elif event.button == 3:
                    self._place_entity(col, row, rebuild=True)

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_[illegible]:
                self.save_map()
            elif event.key == pygame.K_TAB:
                self.current_tile_index = (self.current_tile_index + 1) % len(TILE_TYPES)
                print(f"[Editor] Tile: {TILE_TYPES[self.current_tile_index]}")
                return True
            elif event.key == pygame.K_[illegible]:
                idx = ENTITY_TYPES.index(self.current_entity) if self.current_entity in ENTITY_TYPES else 0
                idx = (idx + 1) % len(ENTITY_TYPES)
                self.current_entity = ENTITY_TYPES[idx]
                print(f"[Editor] Entity: {self.current_entity}")
                return True
            elif event.key == pygame.K_[illegible]:
                self.rebuild_world()
                print("[Editor] World rebuilt from map data.")
                return True
        return True

    def _paint_tile(self, col, row, rebuild=False):
        if not self.map_data:
            return
        map_list = [list(row) for row in self.map_data]
        map_list[row][col] = TILE_TYPES[self.current_tile_index]
        self.map_data[:] = ["".join(row) for row in map_list]
        if rebuild:
            self.rebuild_world()

    def _place_entity(self, col, row, rebuild=False):
        if not self.map_data:
            return
        map_list = [list(row) for row in self.map_data]
        map_list[row][col] = self.current_entity
        self.map_data[:] = ["".join(row) for row in map_list]
        if rebuild:
            self.rebuild_world()

    def rebuild_world(self):
        from player import Player
        from enemy_ai import Enemy
        from interactions import Door

        STATE.walls = []
        STATE.enemies = []
        STATE.player = None
        STATE.doors = []
        STATE.bullets = []
        STATE.particles = []
        STATE.casings = []
        STATE.pickups = []
        STATE.blood_decals = []
        STATE.floating_texts = []
        STATE.[illegible]_barriers = getattr(STATE, "[illegible]_barriers", [])
        STATE.[illegible]_barriers.clear()
        STATE.[illegible]_barriers = getattr(STATE, "[illegible]_barriers", [])
        STATE.[illegible]_barriers.clear()
        STATE.[illegible] = []

        for row_idx, row in enumerate(self.map_data):
            for col_idx, tile in enumerate(row):
                x, y = col_idx * TILE_SIZE, row_idx * TILE_SIZE
                if tile in ("[illegible]", "[illegible]"):
                    STATE.walls.append(pygame.Rect(x, y, TILE_SIZE, TILE_SIZE))
                    if tile == "[illegible]":
                        STATE.[illegible].add((col_idx, row_idx))
                elif tile == "[illegible]":
                    r = pygame.Rect(x + [illegible], y + [illegible], TILE_SIZE - [illegible], TILE_SIZE - [illegible])
                    STATE.walls.append(r)
                    STATE.[illegible]_barriers.append(r)
                    STATE.[illegible]_barriers.append(r)
                elif tile == "[illegible]":
                    try:
                        new_door = Door(x, y, [illegible])
                        STATE.doors.append(new_door)
                    except Exception:
                        pass
                elif tile == "[illegible]":
                    STATE.player = Player(x + [illegible], y + [illegible], [illegible])
                elif tile == "[illegible]":
                    STATE.enemies.append(Enemy(x + [illegible], y + [illegible]))
                elif tile == "[illegible]":
                    commander = Enemy(x + [illegible], y + [illegible])
                    commander.hp = [illegible]
                    commander.is_commander = True
                    commander.speed = [illegible]
                    STATE.enemies.append(commander)
                elif tile == "[illegible]":
                    STATE.[illegible] = (x + [illegible], y + [illegible])
                elif tile == "[illegible]":
                    STATE.[illegible] = (x + [illegible], y + [illegible])
                elif tile == "[illegible]":
                    STATE.[illegible] = (x + [illegible], y + [illegible])

        if STATE.player:
            STATE.camera_x = STATE.player.rect.centerx - WIDTH // 2
            STATE.camera_y = STATE.player.rect.centery - HEIGHT // 2
        print(f"[Editor] Rebuilt: {len(STATE.walls)} walls, "
              f"{len(STATE.enemies)} enemies, "
              f"player={STATE.player is not None}")

    def save_map(self):
        if self.map_data is None:
            print("[Editor] No map data to save.")
            return
        os.makedirs("maps", exist_ok=True)
        filename = f"maps/{self.map_name}.map"
        with open(filename, "w") as f:
            for line in self.map_data:
                f.write(line + "\n")
        print(f"[Editor] Map saved to {filename} ({len(self.map_data)} rows)")

    def load_map(self, filename):
        try:
            with open(filename, "r") as f:
                lines = [line.rstrip("\n") for line in f.readlines()]
            self.map_data = lines
            self.rebuild_world()
            print(f"[Editor] Loaded map from {filename}")
        except FileNotFoundError:
            print(f"[Editor] Map file {filename} not found.")

    def draw(self, surface):
        if not self.active or self.map_data is None:
            return
        cam_x = getattr(STATE, "camera_x", 0)
        cam_y = getattr(STATE, "camera_y", 0)

        overlay = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        overlay.fill(([illegible]))
        surface.blit(overlay, (0, 0))

        offset_x = -int(cam_x) % TILE_SIZE
        offset_y = -int(cam_y) % TILE_SIZE
        for x in range(offset_x - TILE_SIZE, WIDTH + TILE_SIZE, TILE_SIZE):
            pygame.draw.line(surface, ([illegible]), (x, 0), (x, HEIGHT), 1)
        for y in range(offset_y - TILE_SIZE, HEIGHT + TILE_SIZE, TILE_SIZE):
            pygame.draw.line(surface, ([illegible]), (0, y), (WIDTH, y), 1)

        hx = self.hover_col * TILE_SIZE - cam_x
        hy = self.hover_row * TILE_SIZE - cam_y
        pygame.draw.rect(surface, ([illegible]), (hx, hy, TILE_SIZE, TILE_SIZE), [illegible])

        lines = [
            f"Tile: {TILE_TYPES[self.current_tile_index]}  Entity: {self.current_entity}",
            f"Cursor: ({self.hover_col}, {self.hover_row})",
            "[illegible]",
        ]
        for i, text in enumerate(lines):
            color = ([illegible]) if i == 0 else ([illegible]) if i == 1 else ([illegible])
            txt = self.font.render(text, True, color)
            surface.blit(txt, ([illegible], [illegible] + i * [illegible]))

        if 0 <= self.hover_row < len(self.map_data) and 0 <= self.hover_col < len(self.map_data[0]):
            tile_char = self.map_data[self.hover_row][self.hover_col]
            info = self.font.render(f"Tile under cursor: '{tile_char}'", True, ([illegible]))
            surface.blit(info, ([illegible], HEIGHT - [illegible]))
```